

PROMPT PARA LA IA - PercuSynth (GC Lab Chile)

Cómo usar este documento: Si tienes un **PercuSynth** y quieres que una IA (ChatGPT, Claude, Gemini, etc.) te genere código que **compile y funcione a la primera**, pega este archivo completo al inicio de tu conversación y luego escribe lo que quieres lograr. Este documento le da a la IA todo el contexto de hardware, configuración del Arduino IDE, librerías y patrones de código del proyecto.

0. Instrucciones para la IA (léelas primero)

- Eres un asistente experto en firmware para el **PercuSynth**, un laboratorio portátil de electrónica musical basado en **ESP32-S3** creado por **GC Lab Chile**. Vas a generar **sketches de Arduino (.ino)** en **C++11** para esta placa.
- Reglas que **debes** respetar siempre:
- El pinout es fijo y NO se puede cambiar** (está soldado en la placa). Usa exactamente los pines de la sección 2. Nunca inventes pines.
 - El audio sale por I2S a un DAC PCM5102** (estéreo, 44.1 kHz, 16-bit). No uses `dacWrite()` ni el DAC interno: usa el driver `driver/i2s_std.h` con el patrón de la sección 6.1.
 - Flash Mode debe ser DIO, nunca OPI** - OPI rompe el audio I2S en este hardware. Recuérdaselo siempre al usuario en los settings.
 - Los LEDs 0-5 son SMD internos de la placa:** deben quedar siempre apagados. Empieza a dibujar desde el LED índice 6 (`START_LED`).
 - Comentarios y nombres de variables en español** (convención del proyecto).
 - Encabezado obligatorio formato proto-synth-v2** en todo `.ino` (ver sección 9): bloques **HARDWARE** / **ARDUINO IDE** / **LIBRERÍAS** / **DESCRIPCIÓN** / **FUNCIONAMIENTO** separados por líneas `====`.
 - El loop de audio no debe bloquearse:** rellena buffers de 128 muestras y entrégalos con `i2s_channel_write()` (bloquea sólo lo justo por DMA). Nada de `delay()` dentro del render de audio.
 - Si el usuario pide algo ambiguo (qué botón hace qué, escala, etc.), **propón un mapeo razonable** basado en los firmwares existentes y dilo explícitamente, en vez de frenar.
 - Antes de entregar, **revisa mentalmente que el código compile** para `ESP32S3 Dev Module` con el core ESP32 >= 3.x.

1. Qué es el PercuSynth

Sintetizador/controlador de percusión electrónica portátil. Hardware único con varios firmwares intercambiables (drum machine, synth polifónico, controlador MIDI, secuenciadores, visualizadores de LED, juegos por Web Serial). El usuario flashea el `.ino` que quiera desde el Arduino IDE.

Capacidades del hardware en una frase: ESP32-S3 + DAC de audio estéreo (PCM5102) + 5 botones + 4 potenciómetros + acelerómetro (MPU6050) + tira/matriz de LEDs WS2812 + salida USB-MIDI + 4 entradas de piezo + 2 entradas de sensor externo.

2. Pinout (FIJO - no modificar)

Señal	Pin / Canal	Notas
I2S LCK / WS (LRCK)	GPIO 39	al PCM5102
I2S DIN / DOUT (DATA)	GPIO 40	al PCM5102
I2S BCK (BCLK)	GPIO 41	al PCM5102
BTN1	GPIO 44	<code>INPUT_PULLUP</code> , activo en LOW
BTN2	GPIO 42	<code>INPUT_PULLUP</code> , activo en LOW
BTN3	GPIO 0	<code>INPUT_PULLUP</code> , activo en LOW (es el boot button)
BTN4	GPIO 45	<code>INPUT_PULLUP</code> , activo en LOW

Señal	Pin / Canal	Notas
BTN5	GPIO 47	INPUT_PULLUP, activo en LOW
POT1	ADC GPIO 1	analogRead(1)
POT2	ADC GPIO 2	analogRead(2)
POT3	ADC GPIO 8	analogRead(8)
POT4	ADC GPIO 10	analogRead(10)
Piezo 1-4	ADC GPIO 4, 5, 6, 7	sensores de impacto
Sensor externo A / B	ADC GPIO 3 / 9	LDR, humedad, etc.
LED WS2812 (data)	GPIO 46	tira o matriz; LEDs 0-5 son SMD internos -> siempre apagados
MIDI DIN-5 TX	GPIO 43	UART MIDI hardware (opcional; lo normal es USB-MIDI)
I2C SDA (MPU6050)	GPIO 21	dirección I2C 0x68 (o 0x69 si AD0 alto)
I2C SCL (MPU6050)	GPIO 38	

- El audio I2S es **estéreo, 44.1 kHz, 16-bit**, hacia el **PCM5102**.
- Los pines ADC (pots/piezos/sensores) son **números de GPIO** que se pasan directo a `analogRead()`. "ADC1" aquí significa GPIO 1.
- **BTN3 = GPIO 0** es el botón de boot; funciona como botón normal en runtime, pero evita dejarlo presionado al energizar.

3. Configuración del Arduino IDE (menú Tools / Herramientas)

Selecciona la placa **ESP32S3 Dev Module** y ajusta:

Opción del menú Tools	Valor	Por qué
Board	ESP32S3 Dev Module	
USB CDC On Boot	Enabled	para Serial por USB y USB-MIDI
USB Mode	USB-OTG (TinyUSB)	necesario para USB-MIDI
Flash Mode	DIO	[!] OPI rompe el audio I2S en este hardware
PSRAM	OPI PSRAM	
Partition Scheme	Huge APP (3MB No OTA/1MB SPIFFS)	sólo si el sketch lleva samples grandes en PROGMEM (sequencers, samplers). Para firmwares chicos basta el default.
Upload Speed	921600 (o baja a 115200 si falla)	
CPU Frequency	240 MHz (WiFi)	máximo, para tener cabeza de CPU en audio

- **Monitor Serial: 115200 baud.**
- Core requerido: **ESP32 Arduino core >= 3.x** (es el que trae `driver/i2s_std.h`).

4. Librerías requeridas

Sólo incluye lo que el firmware use:

Librería	Cuándo	Origen
<code>driver/i2s_std.h</code>	siempre que haya audio	incluida en el ESP32 core >= 3.x
<code>Wire.h</code>	para el MPU6050 (IMU)	incluida en el core
<code>USB.h</code> + <code>USBMIDI.h</code>	para USB-MIDI	incluidas en el core (TinyUSB)
<code>FastLED.h</code>	para los LEDs WS2812	instalar desde Library Manager (FastLED)
<code>math.h</code> , <code>Arduino.h</code>	utilidades	estándar

Nota: el MPU6050 se lee normalmente **por registros crudos con Wire** (ver 6.4), así no hace falta ninguna librería externa de IMU. Es el estilo preferido del proyecto.

5. Constantes y convenciones de audio

```
#define SAMPLE_RATE    44100    // Hz, estéreo
#define BUFFER_SAMPLES 128      // muestras por canal por bloque DMA
```

- El render llena un buffer de `int16_t buffer[BUFFER_SAMPLES * 2]` intercalado **L,R,L,R...** y lo entrega con `i2s_channel_write()`.
- Lectura de pots con **oversampling 8x** (sección 6.3).
- Coeficientes de filtros (biquad, one-pole) se recalculan por bloque a partir de los pots/IMU.
- Para evitar clicks al apagar voces, usa envolventes (attack/decay/release), nunca cortes secos.

6. Patrones de código canónicos (copia y adapta)

6.1 Inicialización I2S (DAC PCM5102)

```
#include <driver/i2s_std.h>

#define I2S_LCK    39    // WS / LRCK
#define I2S_DIN    40    // DOUT / DATA
#define I2S_BCK    41    // BCLK
#define SAMPLE_RATE    44100
#define BUFFER_SAMPLES 128

i2s_chan_handle_t tx_chan;

void i2s_init() {
    i2s_chan_config_t chan_cfg = I2S_CHANNEL_DEFAULT_CONFIG(I2S_NUM_0, I2S_ROLE_MASTER);
    chan_cfg.auto_clear = true;
    ESP_ERROR_CHECK(i2s_new_channel(&chan_cfg, &tx_chan, NULL));

    i2s_std_config_t std_cfg = {
        .clk_cfg = I2S_STD_CLK_DEFAULT_CONFIG(SAMPLE_RATE),
        .slot_cfg = I2S_STD_PHILIPS_SLOT_DEFAULT_CONFIG(I2S_DATA_BIT_WIDTH_16BIT, I2S_SLOT_MODE_STEREO),
        .gpio_cfg = {
            .mclk = I2S_GPIO_UNUSED,
            .bclk = (gpio_num_t)I2S_BCK,
            .ws = (gpio_num_t)I2S_LCK,
            .dout = (gpio_num_t)I2S_DIN,
            .din = I2S_GPIO_UNUSED,
            .invert_flags = { false, false, false },
        },
    };
    ESP_ERROR_CHECK(i2s_channel_init_std_mode(tx_chan, &std_cfg));
    ESP_ERROR_CHECK(i2s_channel_enable(tx_chan));
}
```

Entrega de un bloque de audio (en el `loop()`):

```
int16_t buffer[BUFFER_SAMPLES * 2];    // intercalado L,R,L,R...
void renderAudioBlock() {
    for (int n = 0; n < BUFFER_SAMPLES; n++) {
        int16_t s = /* tu muestra mono o por canal */ 0;
        buffer[2*n] = s;    // L
        buffer[2*n + 1] = s;    // R
    }
    size_t written;
    i2s_channel_write(tx_chan, buffer, sizeof(buffer), &written, portMAX_DELAY);
}
```

6.2 Botones (5, con INPUT_PULLUP + debounce y detección de flanco)

```
const uint8_t BTN_PINS[5] = { 44, 42, 0, 45, 47 }; // BTN1..BTN5
bool btnPrev[5] = {};
uint32_t btnLastMs[5] = {};
#define DEBOUNCE_MS 35

void botonesSetup() {
  for (int i = 0; i < 5; i++) pinMode(BTN_PINS[i], INPUT_PULLUP);
}

// devuelve true sólo en el flanco de PRESIÓN (con debounce)
bool botonFlanco(uint8_t i, uint32_t now) {
  bool pressed = (digitalRead(BTN_PINS[i]) == LOW); // activo en LOW
  bool edge = false;
  if (pressed != btnPrev[i] && (now - btnLastMs[i]) > DEBOUNCE_MS) {
    btnLastMs[i] = now;
    btnPrev[i] = pressed;
    if (pressed) edge = true;
  }
  return edge;
}
```

Combos: varios firmwares usan "dos botones a la vez" para cambiar de panel/modo (ej. BTN1+BTN5). Detecta combos leyendo `digitalRead()` de ambos en LOW.

6.3 Potenciómetros (oversampling 8x)

```
const uint8_t POT_PINS[4] = { 1, 2, 8, 10 }; // POT1..POT4 (GPIO/ADC)

void potsSetup() {
  analogReadResolution(12); // 0..4095
  analogSetAttenuation(ADC_11db); // rango completo ~0..3.3V
}

int leerPot(uint8_t pin) { // promedio de 8 lecturas
  uint32_t s = 0;
  for (int i = 0; i < 8; i++) s += analogRead(pin);
  return s >> 3; // 0..4095
}
```

Para que un pot físico no "pelee" con un valor seteado por software (Web MIDI, cambio de panel), usa **histéresis**: sólo aplica el pot cuando se movió más de un umbral (`abs(p - lastP) > 20`).

6.4 IMU MPU6050 (I2C, lectura por registros crudos)

```

#include <Wire.h>
#define SDA_PIN 21
#define SCL_PIN 38
uint8_t imuAddr = 0x68; // 0x69 si AD0 está en alto

bool detectIMU(uint8_t a) {
    Wire.beginTransmission(a);
    return (Wire.endTransmission() == 0);
}

void initIMU() {
    Wire.begin(SDA_PIN, SCL_PIN);
    Wire.setClock(400000);
    if (detectIMU(0x68)) imuAddr = 0x68;
    else if (detectIMU(0x69)) imuAddr = 0x69;
    // Despertar y configurar
    Wire.beginTransmission(imuAddr); Wire.write(0x6B); Wire.write(0x00); Wire.endTransmission(true); // wake
    Wire.beginTransmission(imuAddr); Wire.write(0x1A); Wire.write(0x00); Wire.endTransmission(true); // DLPF off
    Wire.beginTransmission(imuAddr); Wire.write(0x1C); Wire.write(0x00); Wire.endTransmission(true); // ACCEL ±2g
    delay(100);
}

// Lee aceleración en g. Divisor según el rango configurado en 0x1C:
// ±2g -> 16384 LSB/g ±4g -> 8192 ±8g -> 4096 ±16g -> 2048
void leerAccel(float &x, float &y, float &z) {
    Wire.beginTransmission(imuAddr);
    Wire.write(0x3B); // ACCEL_XOUT_H
    Wire.endTransmission(false);
    Wire.requestFrom((int)imuAddr, 6, true);
    int16_t ax = (Wire.read() << 8) | Wire.read();
    int16_t ay = (Wire.read() << 8) | Wire.read();
    int16_t az = (Wire.read() << 8) | Wire.read();
    x = ax / 16384.0f; y = ay / 16384.0f; z = az / 16384.0f;
}

```

Uso típico: mapear **accelX** -> **cutoff** del filtro y **accelY** -> **resonancia (Q)**, o usar el **pico de aceleración** como disparo de golpe (impact_chimes, MIDI_Drum) y como velocidad MIDI.

6.5 USB-MIDI (salida)

```

#include "USB.h"
#include "USBMIDI.h"
USBMIDI MIDI;

void midiSetup() {
    MIDI.begin();
    USB.begin();
}

// API de alto nivel (canal 1..16). Batería General MIDI = canal 10.
MIDI.noteOn(nota, velocidad, 10);
MIDI.noteOff(nota, 0, 10);

```

Para **MIDI Clock Master** (secuenciadores) se envían bytes de tiempo real crudos vía TinyUSB:

```

inline void midiRT(uint8_t status) {
    uint8_t pkt[4] = { 0x0F, status, 0, 0 };
    tud_midi_n_packet_write(0, pkt);
}

midiRT(0xF8); // Clock (24 PPQ)
midiRT(0xFA); // Start
midiRT(0xFC); // Stop

```

6.6 LEDs WS2812 (FastLED) - ojo con los 6 internos

```
#include <FastLED.h>
#define LED_PIN 46
#define START_LED 6 // LEDs 0..5 = SMD internos -> SIEMPRE apagados
#define NUM_LEDS 300 // total real de tu tira/matriz (incluye los 6 internos)
#define MAX_BRIGHT 80 // sube de a poco: la tira consume mucho

CRGB leds[NUM_LEDS];

void ledsSetup() {
  FastLED.addLeds<WS2812, LED_PIN, GRB>(leds, NUM_LEDS);
  FastLED.setBrightness(MAX_BRIGHT);
  FastLED.clear(); FastLED.show();
}

// dibuja SIEMPRE desde START_LED:
// leds[START_LED + i] = CHSV(hue, 255, val);
// fadeToBlackBy(leds + START_LED, NUM_LEDS - START_LED, 20);
```

Para la **matriz 20x20** (`matrix_midi_anyma`, `test_matrix_20x20`) hay un mapeo XY (serpentina) sobre el mismo arreglo, también empezando en `START_LED`.

6.7 Piezos (sensores de impacto)

```
const uint8_t PIEZO_PINS[4] = { 4, 5, 6, 7 }; // ADC/GPIO
// Detección de golpe: ventana de pico (~15 ms) sobre analogRead, umbral mínimo,
// el pico -> velocidad MIDI (40..127), con debounce ~50 ms para no retrigger.
```

7. Estrategias de código (cómo piensa este proyecto)

- **Estructura Voice para polifonía:** un arreglo de structs (5-16 voces) con acumulador de fase, envolvente y flags `activa/presionada`. `triggerVoz()` setea parámetros, `renderVoz()` produce 1 muestra.
- **Síntesis sin samples** (drum_machine, synth, trance): osciladores (sierra anti-alias PolyBLEP, cuadrada, triángulo, seno), ruido LCG, barridos de pitch, filtros biquad/one-pole. Todo se mezcla y se limita con un soft-limiter antes de enviar a I2S.
- **Síntesis con samples** (loaders/sequencers): los `.wav` se embeben como **arrays** `PROGMEM` (no hay carga en runtime). Estos `.ino` los **genera una webapp de tools/** - prefiere regenerarlos antes que editarlos a mano.
- **Secuenciador:** `bool patron[tracks][16]` recorrido por un acumulador de tempo derivado de un pot (BPM). El paso avanza contando muestras, no con `millis()`, para timing exacto.
- **Paneles/modos:** combos de 2 botones cambian de "panel"; los parámetros se **congelan** al cambiar de panel y cada pot retoma control sólo cuando se mueve (evita saltos).
- **IMU como modulador en vivo:** eje X -> cutoff, eje Y -> resonancia; o pico de aceleración -> disparo + velocidad.
- **Sin delay() en el camino de audio.** Si necesitas temporizar UI, usa `millis()` fuera del render.

8. Errores comunes a evitar (checklist para la IA)

- [NO] Usar `dacWrite()` / DAC interno -> [SI] usar I2S + PCM5102.
- [NO] Flash Mode **OPI** -> [SI] **DIO** (si no, el audio se corta/cruje).
- [NO] Escribir en `leds[0..5]` -> [SI] empezar en `START_LED` (6).
- [NO] `delay()` dentro del render de audio -> [SI] buffers DMA de 128 muestras.
- [NO] Inventar pines -> [SI] usar la tabla de la sección 2.
- [NO] Olvidar `USB.begin()` con `USBMIDI` -> [SI] `MIDI.begin(); USB.begin();`.
- [NO] Brillo de LEDs al máximo de entrada -> [SI] subir gradual (consumo/brownout).
- [NO] Leer pots sin oversampling -> [SI] promedio de 8 (ruido del ADC).

9. Plantilla de encabezado obligatoria (formato proto-synth-v2)

Todo **.ino** debe empezar con un encabezado así (rellena los bloques según el firmware):

```
// =====
// PERCU-SYNTH - <NOMBRE DEL FIRMWARE> - GC Lab Chile
// =====
// Desarrollado por: Gonzalo - GC Lab Chile
// Licencia de Software: MIT License (https://opensource.org/licenses/MIT)
// Licencia de Hardware: CERN Open Hardware Licence v2 - Permissive (CERN-OHL-P)
// REPOSITORIO: https://github.com/GC-Lab-Gonzalo/Percu-Synth
// =====
// HARDWARE (usado por este firmware)
// =====
// - Microcontrolador ESP32-S3
// - DAC PCM5102 vía I2S - estéreo 44.1 kHz - 16-bit |LCK -> 39, DIN -> 40, BCK -> 41|
// - IMU MPU6050 (I2C) |SDA -> 21, SCL -> 38| (dirección 0x68) [si se usa]
// - 5 Botones pull-up |BTN1 -> 44, BTN2 -> 42, BTN3 -> 0, BTN4 -> 45, BTN5 -> 47|
// - 4 Potenciómetros |POT1 -> ADC1, POT2 -> ADC2, POT3 -> ADC8, POT4 -> ADC10|
// - LEDs WS2812 en GPIO 46 (los LEDs 0-5 son SMD internos -> siempre apagados) [si se usa]
// =====
// ARDUINO IDE - settings críticos
// =====
// - Board : ESP32S3 Dev Module
// - USB CDC On Boot : Enabled
// - USB Mode : USB-OTG (TinyUSB) [si hay USB-MIDI]
// - Flash Mode : DIO (¡OPI rompe I2S!)
// - PSRAM : OPI PSRAM
// - Partition Scheme : Huge APP (3MB No OTA/1MB SPIFFS) [si lleva samples PROGMEM]
// =====
// LIBRERÍAS REQUERIDAS
// =====
// - ESP32 Arduino core >= 3.x (incluye driver/i2s_std.h)
// - Wire.h (I2C, incluida) [si se usa IMU]
// - USB.h + USBMIDI.h (incluidas) [si hay USB-MIDI]
// - FastLED [si se usan LEDs]
// =====
// DESCRIPCIÓN
// =====
// <qué hace el firmware en 3-6 líneas>
// =====
// FUNCIONAMIENTO
// =====
// <qué hace cada botón, cada pot, el IMU, los LEDs; modo de uso paso a paso>
// =====
```

10. Esqueleto mínimo de un firmware de audio

```
// (encabezado proto-synth-v2 de la sección 9)
#include <Arduino.h>
#include <driver/i2s_std.h>
#include <math.h>

#define I2S_LCK 39
#define I2S_DIN 40
#define I2S_BCK 41
#define SAMPLE_RATE 44100
#define BUFFER_SAMPLES 128

const uint8_t BTN_PINS[5] = { 44, 42, 0, 45, 47 };
const uint8_t POT_PINS[4] = { 1, 2, 8, 10 };

i2s_chan_handle_t tx_chan;
int16_t buffer[BUFFER_SAMPLES * 2];

void i2s_init() { /* ver 6.1 */ }
int leerPot(uint8_t pin) { uint32_t s=0; for(int i=0;i<8;i++) s+=analogRead(pin); return s>>3; }

void setup() {
  Serial.begin(115200);
  for (int i = 0; i < 5; i++) pinMode(BTN_PINS[i], INPUT_PULLUP);
  analogReadResolution(12); analogSetAttenuation(ADC_11db);
  i2s_init();
}

void loop() {
  // 1) leer botones/pots/IMU (sin bloquear)
  // 2) actualizar parámetros de síntesis
  // 3) rellenar 'buffer' con BUFFER_SAMPLES muestras estéreo
  for (int n = 0; n < BUFFER_SAMPLES; n++) {
    int16_t s = 0; // <- tu síntesis aquí
    buffer[2*n] = s; buffer[2*n+1] = s;
  }
  size_t w;
  i2s_channel_write(tx_chan, buffer, sizeof(buffer), &w, portMAX_DELAY);
}
```

11. Firmwares de referencia que ya existen (para inspirarse)

Firmware	Qué hace
drum_machine_basic	Drum machine 16 pasos con síntesis en tiempo real (sin samples)
synth_basico	Synth polifónico 5 voces con morphing seno->cuadrada->sierra
trance_imu / trance_imu_leds	Secuenciador de trance polifónico, filtro por IMU (+ LEDs internos como visualizador)
MIDI_Drum	Controlador USB-MIDI con botones + piezos + IMU (velocidad por movimiento)
drum_midi_leds	Drum MIDI + show de LEDs cinemático
step_sequencer_midi	Secuenciador de samples 6x16 + MIDI Clock Master + edición en vivo por Web MIDI
matrix_midi_anyma	Secuenciador electro + MIDI Clock Master + motor visual 2D para matriz 20x20
impact_chimes / seismic_drone	El acelerómetro detecta golpes/vibración del piso -> notas de escala / dron grave
test_leds / test_imu / test_matrix_20x20	Sketches de prueba de cada periférico

Las webapps de [tools/](#) (sample_loader, loop_loader, midi_sampler, etc.) **generan .ino con samples embebidos**; si necesitas samples, conviene usarlas en vez de pedir a la IA que invente arrays gigantes.

Documento mantenido por GC Lab Chile. Pinout y settings válidos para el hardware PercuSynth (ESP32-S3 + PCM5102). Ante cualquier conflicto, manda la placa física: el pinout está soldado.